Mardi 20/10 — Tableaux et chaînes de caractères

Kevin Bardoux

mardi 20 octobre 2026

Table des matières

Objectifs 2
Matin — Tableaux & chaînes de caractères — exercices 2
Rappel (5 min) 2
Apport 1 — Les tableaux (10 min) 2
Tableaux — attention et vérification 4
Exercices t1 à t3 — Tableaux (35 min) 4
Apport 2 — Les chaînes de caractères (15 min) 5
Chaînes — attention et vérification 6
Exercices t4 et t5 — Chaînes (30 min) 6
Si le temps le permet — t6 (mot interdit) 7
À retenir — ce matin 7
Débrief 7
Après-midi — TP — « Problem set 2 » (validé par les tests) 8
Consignes (1 h 55) 8
Les six fonctions 8
Si le temps le permet — p2-masquer-carte, puis un vrai palindrome 9
Pour aller plus loin 9
Sources 9

> **Note**
>
> **Compétences visées** : résolution de problème **Livrable** : les cinq premières fonctions du Problem set 2 au vert (la sixième, `p2-masquer-carte`, mercredi après-midi si besoin).

Objectifs

- Lire, ajouter et parcourir les valeurs d'un tableau.
- Parcourir une chaîne et utiliser ses méthodes courantes.
- Renvoyer un **nouveau** tableau sans modifier celui reçu.
- Faire passer les tests du Problem set 2.

Comment se passe la journée. Le matin, deux notions : le tableau, la chaîne de caractères. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine la matinée. L'après-midi, le Problem set 2.

Matin — Tableaux & chaînes de caractères — exercices

Rappel (5 min)

1. Différence entre `console.log` et `return` ?
2. Que signale une `ReferenceError` ?
3. Que donne `typeof '42'` ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — Les tableaux (10 min)

- **Pourquoi** : garder plusieurs valeurs sous un seul nom.
- **Tableau** : une liste ordonnée ; le premier index est `0`.

Pourquoi. Trois notes, cent prénoms, une liste de musées : créer une variable par valeur serait impossible. Un tableau les range sous un seul nom, dans l'ordre.

Définition. Un tableau est une liste ordonnée de valeurs, repérées par un **index** qui commence à 0 (MDN Web Docs, s. d.-a). `length`

donne le nombre de valeurs ; le dernier index est donc `length - 1`. La boucle `for...of` parcourt les valeurs une à une (MDN Web Docs, s. d.-e) ; `push` ajoute un ou plusieurs éléments à la fin (MDN Web Docs, s. d.-c).

Exemple.

```
const notes = [12, 15, 9];

console.log(notes[0]);     // 12
console.log(notes.length); // 3
notes.push(18);            // [12, 15, 9, 18]

for (const note of notes) {
  console.log(note);
}
```

Pas à pas. Le formateur tape chaque ligne dans le REPL ; vous la retapez aussitôt.

1. Tapez `const notes = [12, 15, 9]`, puis Entrée.
2. Tapez `notes[0]` : `12`, la première valeur. Puis `notes[2]` : `9`, la dernière.
3. Tapez `notes.length` : `3`. Le dernier index est `3 - 1 = 2`.
4. Tapez `notes.push(18)` : `4`, le nouveau nombre de valeurs. Puis `notes` : `[ 12, 15, 9, 18 ]`.
5. Tapez `for (const note of notes) { console.log(note) }` : les quatre notes s'affichent, une par ligne.

Vous devez voir :

```
PS C:\Users\stagiaire\dwwm\semaine-03> node
Welcome to Node.js v24.21.0.
Type ".help" for more information.
> const notes = [12, 15, 9]
undefined
> notes[0]
12
> notes[2]
```

```
9
> notes.length
3
> notes.push(18)
4
> notes
[ 12, 15, 9, 18 ]
> for (const note of notes) { console.log(note) }
12
15
9
18
undefined
```

`undefined` après `const …` ou après la boucle : l'instruction ne renvoie pas de valeur. Ignorez-le.

Une constante `const` peut contenir un tableau que l'on modifie : c'est le **nom** qui ne peut pas désigner un autre tableau (l'étiquette de la semaine 2), pas le contenu du tableau qui ne peut pas changer.

Tableaux — attention et vérification

> ⚠ Attention
>
> `notes[notes.length]` vaut `undefined` : il n'y a rien après la dernière case. Le dernier élément est `notes[notes.length - 1]`.

- **Vérifiez** : avec `['a', 'b', 'c', 'd']`, que vaut l'index 3 ?

Exercices t1 à t3 — Tableaux (35 min)

1. Ensemble (5 min) : **t1-dernier**, le dernier élément, `null` si vide.
2. Seuls : **t2-somme**, la somme, avec `for...of`.
3. Seuls : **t3-pairs**, un **nouveau** tableau des nombres pairs.
4. Chacun : ses tests, puis `npx eslint` suivi de son fichier.
5. **Réussi si** : au vert, et le tableau reçu jamais modifié.

Un test vérifie que le tableau reçu n'est pas modifié : pour t3, créez un tableau vide **avant** la boucle (`const result = [];`), ajoutez-y les nombres pairs avec `push`, puis renvoyez-le. Un nombre est pair si `n % 2 === 0` (le reste, rappelé lundi).

Apport 2 — Les chaînes de caractères (15 min)

- **Pourquoi** : transformer un texte (majuscules, découpe, recherche).
- **Chaîne** : un texte, lu comme un tableau de caractères.

Pourquoi. Un prénom à mettre en majuscules, des initiales à extraire, un mot à chercher dans un message : une application manipule du texte en permanence.

Définition. Une chaîne de caractères (*string*) est un texte. Elle se lit comme un tableau : `word[0]` est le premier caractère, `word.length` le nombre de caractères. Ses méthodes renvoient un **nouveau** texte, sans modifier l'ancien : `toUpperCase` met en majuscules (MDN Web Docs, s. d.-h) ; `includes` indique si un texte en contient un autre (MDN Web Docs, s. d.-f) ; `split` découpe un texte en tableau selon un séparateur (MDN Web Docs, s. d.-g), et `join` recolle un tableau en texte (MDN Web Docs, s. d.-b).

Exemple. Résultats vérifiés avec Node.js :

```
const word = 'Bonjour';

console.log(word[0]);                // 'B'
console.log(word.length);            // 7
console.log(word.toUpperCase());     // 'BONJOUR'
console.log(word.includes('jour')); // true
console.log('a b c'.split(' '));     // ['a', 'b', 'c']
```

Pas à pas. Le formateur tape chaque ligne dans le REPL ; vous la retapez aussitôt.

1. Tapez `const word = 'Bonjour'`.
2. Tapez `word[0]` : `'B'`. Puis `word.length` : `7`.
3. Tapez `word.toUpperCase()` : `'BONJOUR'`. Puis `word` : toujours `'Bonjour'`, la méthode n'a rien modifié.

4. Tapez `word.includes('jour')` : `true`.
5. Tapez `'Ada Lovelace'.split(' ')` : `[ 'Ada', 'Lovelace' ]`, un tableau de deux mots.
6. Tapez `['A.', 'L.'].join(' ')` : `'A. L.'`, un texte.

Vous devez voir :

```
PS C:\Users\stagiaire\dwwm\semaine-03> node
Welcome to Node.js v24.21.0.
Type ".help" for more information.
> const word = 'Bonjour'
undefined
> word[0]
'B'
> word.length
7
> word.toUpperCase()
'BONJOUR'
> word
'Bonjour'
> word.includes('jour')
true
> 'Ada Lovelace'.split(' ')
[ 'Ada', 'Lovelace' ]
> ['A.', 'L.'].join(' ')
'A. L.'
```

Chaînes — attention et vérification

> ⚠ Attention
>
> `word.toUpperCase()` ne change pas `word` : il renvoie un nouveau texte. Gardez-le : `const loud = word.toUpperCase();`.

- **Vérifiez** : que donne `'chat'.length - 1` ? Et `'chat'[3]` ?

Exercices t4 et t5 — Chaînes (30 min)

1. Ensemble (5 min) : **t4-crier**, `shout('bonjour')` renvoie `'BONJOUR!'`.
2. Seuls : **t5-initiales**, `'Ada Lovelace'` donne `'A. L.'`.

3. Chacun : ses tests, puis `npx eslint` suivi de son fichier.
4. **Réussi si** : tests au vert, aucune alerte.

Indice pour t5 : `split(' ')` découpe en mots ; prenez la première lettre de chaque mot, ajoutez un point ; puis `join(' ')`.

Si le temps le permet — t6 (mot interdit)

1. **t6-mot-interdit** : un texte contient-il un mot ?
2. Sans tenir compte des majuscules.
3. Indice : mettez les deux textes en minuscules avant de comparer.

À retenir — ce matin

> À retenir
>
> - **Tableau** : index de `0` à `length - 1` ; `for...of` parcourt.
> - **Chaîne** : se lit comme un tableau ; ses méthodes renvoient du neuf.
> - **Ne modifiez jamais** le tableau reçu : créez-en un nouveau.

Le détail, notion par notion :

- **Tableau** : une liste ordonnée sous un seul nom ; `tableau[0]` est le premier élément, `tableau[tableau.length - 1]` le dernier ; `push` ajoute à la fin ; `for...of` parcourt les valeurs.
- **Chaîne** : `texte[0]`, `texte.length`, `for...of` comme un tableau ; `toUpperCase`, `includes`, `split` et `join` renvoient un nouveau résultat, sans modifier le texte.
- **Nouveau tableau** : créez `const result = [];` avant la boucle, remplissez-le, renvoyez-le.

Débrief

- Pourquoi le dernier élément est-il à l'index `length - 1` ?
- Quand créer un nouveau tableau plutôt que modifier l'ancien ?
- Quelles méthodes de chaîne avez-vous utilisées ?

Après-midi — TP — « Problem set 2 » (validé par les tests)

Consignes (1 h 55)

1. Six fonctions, fichiers `p2-…`, dans l'ordre ; cinq obligatoires.
2. Avant de coder : deux exemples à la main, dont un cas vide.
3. Un test à la fois ; puis ESLint sur le fichier `p2-…`.
4. Ni `Math.max`, ni `sort`, ni `reverse` : c'est l'exercice.

Les six fonctions

- **p2-moyenne** (facile) : moyenne d'un tableau, `null` si vide.
- **p2-maximum** (facile) : « Parcourir une liste » de la semaine 1.
- **p2-voyelles** (facile) : compter a, e, i, o, u, y.
- **p2-inverser** (moyen) : `'chat'` devient `'tahc'`.
- **p2-palindrome** (moyen) : `'Kayak'` se lit dans les deux sens.
- **p2-masquer-carte** (moyen+, si le temps) : `'**** **** **** 5678'`.

Un palindrome se lit de la même façon de gauche à droite et de droite à gauche (Wikipédia, s. d.). Pour p2-masquer-carte, `slice` extrait une partie d'un tableau (MDN Web Docs, s. d.-d) ; les textes ont une méthode `slice` qui fonctionne de la même façon : `'12345678'.slice(4)` donne `'5678'`. Masquer les numéros de carte est une précaution de sécurité : un site ne doit jamais afficher un numéro complet.

> Indice — p2-masquer-carte
>
> Vérifiez d'abord la longueur (16), puis chaque caractère avec `'0123456789'.includes(caractère)`. Si tout est bon, recollez `'**** **** **** '` et les 4 derniers chiffres.

Critères de réussite

- □ Les tests des cinq premières fonctions `p2-…` sont au vert
- □ ESLint ne signale rien sur ces fichiers
- □ Aucune fonction ne modifie le tableau reçu

Si le temps le permet — p2-masquer-carte, puis un vrai palindrome

1. `p2-masquer-carte` ; sinon, mercredi après-midi.
2. « Ésope reste ici et se repose » est un palindrome, si l'on ignore les espaces.
3. Écrivez une version de p2-palindrome qui ignore aussi les espaces (sans test fourni).

Pour aller plus loin

- Comptez aussi les voyelles accentuées : é, è, ê, à, ù…
- Écrivez `minimumAndMaximum(numbers)` : un tableau `[min, max]`.

Sources

MDN Web Docs. s. d.-a. « Array - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array.

MDN Web Docs. s. d.-b. « Array : méthode join() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/join.

MDN Web Docs. s. d.-c. « Array : méthode push() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/push.

MDN Web Docs. s. d.-d. « Array : méthode slice() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/slice.

MDN Web Docs. s. d.-e. « for...of - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/for...of.

MDN Web Docs. s. d.-f. « String.prototype.includes() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String/includes.

MDN Web Docs. s. d.-g. « String.prototype.split() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String/split.

MDN Web Docs. s. d.-h. « String.prototype.toUpperCase() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String/toUpperCase.

Wikipédia. s. d. « Palindrome ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Palindrome.